

DSi Design & Verification Engineer

Broad Problem-Solving & Numerical Question Bank
with Worked Solutions + 4 Full 60-Minute Mock Tests

Correction to the first bank: This version is deliberately built around multi-part design problems, timing calculations, waveform tracing, K-maps, counters, FSM construction, RTL debugging and testbench design. It is much closer to what a serious 60-minute written engineering screening can test.

Public DSi reports for other technical roles repeatedly describe written rounds as problem-solving oriented, including multi-question paper-based problems rather than only definitions. For this semiconductor role, the confirmed syllabus is Verilog/SystemVerilog, digital logic, FSM and basic RTL/testbench; therefore these problems combine those exact areas.

Problem 1 — K-map + NAND-only implementation [12 marks]

A safety controller has inputs A,B,C,D. Output F must be 1 for minterms m(0,2,5,7,8,10,13,15).

- (a) Minimize F using a 4-variable K-map.
- (b) Express the result in SOP form.
- (c) Implement using NAND gates only.
- (d) If each NAND has 2 ns propagation delay, estimate the worst logic delay of a two-level NAND-NAND implementation.

Worked solution: The minterms form two 4-cell groups: {0,2,8,10} gives $B'D'$; {5,7,13,15} gives BD .

Therefore $F = B'D' + BD = B \text{ XNOR } D$. A and C cancel completely.

NAND-NAND: generate B' and D' with NAND inverters if complements are unavailable, form $\text{NAND}(B',D')$ and $\text{NAND}(B,D)$, then final NAND. If complemented inputs are already available, the core is two logic levels = about 4 ns. If $B'D'$ must also be generated, paths using them add an inverter level, about 6 ns worst-case.

Problem 2 — Full-adder delay / ripple-carry timing [12 marks]

A 4-bit ripple-carry adder is built from identical full adders. For each stage, carry-in to carry-out delay is 2 ns; carry-in to sum delay is 3 ns. Ignore other input delays.

- (a) Find the worst-case time for C4 after C0 changes.
- (b) Find the worst-case time for S3.
- (c) What is the approximate maximum addition rate if the next operation cannot begin until all outputs are valid?
- (d) Explain why a carry-lookahead structure can improve this.

Worked solution: (a) C0 propagates through four carry stages: $4 \times 2 = 8$ ns.

(b) To S3, C0 must propagate through stages 0,1,2 (6 ns) and then through the stage-3 carry-to-sum path (3 ns): 9 ns.

(c) Worst output is S3 at 9 ns, so idealized maximum rate $\approx 1/(9 \text{ ns}) = 111.1 \text{ MHz}$.

(d) Ripple carry is $O(N)$ in carry depth. Carry-lookahead computes generate/propagate relations in parallel/hierarchically, reducing carry depth.

Problem 3 — Setup-time / maximum clock frequency [12 marks]

Two positive-edge registers are separated by combinational logic. $T_{cq}(\text{max}) = 0.8$ ns, $T_{comb}(\text{max}) = 5.6$ ns, $T_{setup} = 0.7$ ns. Worst clock uncertainty/skew allowance is 0.4 ns.

- (a) Calculate minimum clock period.
- (b) Calculate f_{max} .
- (c) If the combinational block is pipelined into two equal 2.8 ns blocks with an added register, estimate new f_{max} using the same register timing.
- (d) What happens to latency?

Worked solution: (a) $T_{\text{clk}} \geq 0.8 + 5.6 + 0.7 + 0.4 = 7.5$ ns.

(b) $f_{\text{max}} \leq 1/7.5 \text{ ns} = 133.33 \text{ MHz}$.

(c) New stage period $\geq 0.8 + 2.8 + 0.7 + 0.4 = 4.7$ ns, so $f_{\text{max}} \approx 212.77 \text{ MHz}$.

(d) Frequency/throughput potential improves, but the operation now crosses an additional pipeline stage, increasing cycle latency.

Problem 4 — Hold-time numerical [10 marks]

For a register-to-register path, $T_{cq}(\text{min}) = 0.25$ ns, $T_{comb}(\text{min}) = 0.15$ ns, destination hold time = 0.55 ns. Assume destination clock can arrive 0.10 ns later than source clock (positive skew harmful for hold).

- (a) Check hold timing using data arrival $\geq$ required hold.
- (b) Find the hold slack.
- (c) How much extra minimum data-path delay is required if it fails?

Worked solution: Earliest data arrival relative to source edge $= 0.25 + 0.15 = 0.40$ ns. With destination edge 0.10 ns later, required absolute stability point is $0.10 + 0.55 = 0.65$ ns. Slack $= 0.40 - 0.65 = -0.25$ ns, so it fails by 0.25 ns. Add at least 0.25 ns minimum delay (practically margin is added too).

Problem 5 — Counter design and width [12 marks]

A timer receives a 50 MHz clock and must produce a one-clock `tick` every 1 ms.

- (a) How many input clocks occur per millisecond?
- (b) What counter modulus is required?
- (c) Minimum counter width?
- (d) Write synthesizable SystemVerilog that generates a one-cycle tick.
- (e) What count value should trigger tick if counting begins at zero?

Worked solution: 50 MHz $\times$ 1 ms = 50,000 clocks. Modulus=50,000. Since $2^{15}=32768$ and $2^{16}=65536$, width=16 bits. Trigger at 49,999 then reset to 0.

Problem 6 — Frequency divider [10 marks]

A 100 MHz clock drives a binary counter.

- (a) What frequency appears at Q0, Q1, Q2 and Q9?
- (b) Which counter bit is closest to 100 kHz?
- (c) If an exact 100 kHz 50%-duty output is required using a toggle flip-flop, after how many input cycles should the output toggle?

Worked solution: Binary counter bit k divides by $2^{(k+1)}$. Q0=50 MHz, Q1=25 MHz, Q2=12.5 MHz, Q9=100 MHz/1024=97.65625 kHz. Q9 is closest to 100 kHz. For exact 100 kHz output, half-period is 5 us; at 100 MHz that is 500 clocks, so toggle every 500 input cycles.

Problem 7 — Blocking vs nonblocking trace [12 marks]

Initially a=1, b=0, c=0. At a rising edge evaluate:

Version X: a = b; b = c; c = a;

Version Y: a <= b; b <= c; c <= a;

- (a) Find (a,b,c) after Version X.
- (b) Find (a,b,c) after Version Y.
- (c) Which better models three registers operating in parallel? Explain.

Worked solution: X uses immediate procedural updates: a=0; b=0; c=a(now 0), so (0,0,0).

Y samples old RHS values: a<-0, b<-0, c<-1, so (0,0,1).

Y/nonblocking models parallel edge-triggered registers.

Problem 8 — RTL bug hunt [15 marks]

Consider:

```
always @(posedge clk) begin
```

```
if (rst) count = 0;
```

```
if (en) count = count + 1;
```

```
end
```

- (a) Identify behavioral/coding problems.
- (b) If rst=1 and en=1 on the same edge, what happens in simulation?
- (c) Rewrite with reset priority.
- (d) Modify it into a saturating 8-bit counter.

Worked solution: Problems: blocking assignments in sequential RTL; two independent if statements mean enable can overwrite reset in the same edge; priority is unclear. With rst=en=1, count becomes 0 then immediately 1. Correct:

```
`if(rst) count<=0; else if(en) count<=count+1;`. Saturating: increment only if `count != 8'hFF`.
```

Problem 9 — Latch inference + truth behavior [12 marks]

Given:

```
always_comb begin
```

```
if (sel==2'b00) y=a;
```

```
else if (sel==2'b01) y=b;
```

```
else if (sel==2'b10) y=c;
```

```
end
```

- (a) What happens for sel=11?

- (b) What hardware may be inferred?
- (c) Correct it so sel=11 outputs d.
- (d) Give a second correction style using a default assignment.

Worked solution: For 11, y has no new assignment and retains previous value, implying storage/latch behavior. Add final ``else y=d;``, or begin with ``y=d;`` and override it for 00/01/10.

Problem 10 — FSM: overlapping sequence detector 1011 [18 marks]

Design a detector for serial sequence 1011; overlaps are allowed.

- (a) Define states by matched prefix.
- (b) Give the complete transition table.
- (c) Produce a Mealy output.
- (d) For input 1011011, identify cycles on which detect asserts.
- (e) Write the next-state/output RTL.

Worked solution: Use S0(no match), S1(1), S10(10), S101(101).

Transitions: S0:0->S0,1->S1. S1:0->S10,1->S1. S10:0->S0,1->S101. S101:0->S10,1->S1 with detect=1 on that '1'.

For 1 0 1 1 0 1 1, detections occur on bits 4 and 7. After detection, final 1 is retained as prefix S1, enabling overlap.

Problem 11 — FSM: vending machine [18 marks]

A product costs 15 units. Machine accepts 5 and 10-unit coins, one coin per clock. It returns change when 20 has been inserted.

- (a) Choose states.
- (b) Build transition/output table as a Mealy machine.
- (c) Determine dispense/change for input sequence 10,10,5,5,5.
- (d) Write synthesizable SystemVerilog next-state/output logic.

Worked solution: States S0,S5,S10. S0+5->S5; S0+10->S10. S5+5->S10; S5+10->S0 with dispense=1. S10+5->S0 dispense=1; S10+10->S0 dispense=1 and change=5.

Sequence: first 10 ->S10; second 10 dispenses+change and S0; then 5->S5; next 5->S10; final 5 dispenses and S0.

Two products total, one change event.

Problem 12 — FSM: handshake controller [16 marks]

A block receives ``start``. It must assert ``busy`` from the next clock until external ``done=1``; then assert ``valid`` for exactly one clock and return IDLE.

- (a) Draw/describe states and transitions.
- (b) Is a 3-state Moore design natural? Why?
- (c) What if start stays high for many cycles?
- (d) Write RTL that does not retrigger until returning to IDLE.

Worked solution: States IDLE, BUSY, VALID. IDLE on start->BUSY; BUSY stays until done then VALID; VALID unconditionally->IDLE. Moore outputs: busy=1 only BUSY; valid=1 only VALID. If start remains high when VALID returns to IDLE, the following IDLE evaluation can start a new operation; if the specification forbids that, add edge detection or require start deassertion/READY protocol.

Problem 13 — State encoding [10 marks]

An FSM has 13 states.

- (a) Minimum bits with binary encoding?
- (b) Flip-flops with one-hot encoding?
- (c) Number of unused binary encodings?
- (d) Give one trade-off.

Worked solution: $\text{ceil}(\log_2 13)=4$ bits. One-hot needs 13 flops. Four-bit binary has 16 codes, so 3 unused. Binary saves flops; one-hot often simplifies decode and can improve FPGA speed but uses more storage.

Problem 14 — Testbench timing / waveform reasoning [15 marks]

A DUT samples `d` on posedge clk into q. Clock starts 0 and toggles every 5 ns. q starts/reset to 0. Reset is released at 12 ns. d changes to 1 at 17 ns, to 0 at 24 ns.

- (a) List rising edges through 35 ns.
- (b) Assuming synchronous active-high reset was high at the 5 ns edge and low after 12 ns, find q just after each rising edge.
- (c) At what edge is d=1 first captured?
- (d) Why is changing d exactly at 25 ns a poor naive testbench practice?

Worked solution: Rising edges: 5,15,25,35 ns. At 5 reset captures q=0. At 15 reset low and d's assumed initial 0 ->q=0. d=1 at17 then d=0 at24, so at25 q captures 0; therefore the 1 pulse was never sampled. At35 q remains0. If d changed exactly at25, DUT and stimulus could race depending on scheduling; drive away from sample edge or use disciplined clocking.

Problem 15 — Self-checking counter testbench [16 marks]

Write a self-checking strategy for a 4-bit up counter with synchronous reset and enable.

- (a) Generate clock.
- (b) Apply reset.
- (c) Check hold when en=0.
- (d) Check at least 20 enabled cycles including wrap 15->0.
- (e) Explain how a reference variable can calculate expected count.

Worked solution: Maintain integer/logic expected. After reset expected=0. On each checked posedge: if reset expected=0; else if en expected=(expected+1)&4'hF. After DUT nonblocking update settles (e.g. #1 in a simple TB), compare `count != expected` and call `\$error`. Explicitly test en=0 and enough enabled cycles to cross wrap.

Problem 16 — Signed arithmetic / overflow [12 marks]

Using 4-bit two's complement:

- (a) Compute 5+4 and state whether signed overflow occurs.
- (b) Compute 7+3 in 4 bits and identify the interpreted signed result.
- (c) Give the signed overflow rule.
- (d) Distinguish overflow from carry-out.

Worked solution: 5=0101,4=0100 ->1001. As 4-bit signed, 1001=-7, so overflow occurred because two positive operands produced a negative-sign result. 7+3=0111+0011=1010=-6 interpreted signed, also overflow. Signed overflow: same-sign operands produce opposite-sign result. Carry-out is primarily an unsigned arithmetic indicator and is not equivalent to signed overflow.

Problem 17 — Processor datapath reasoning [14 marks]

A single-cycle datapath executes `ADD x3,x1,x2` and `LW x5,8(x1)`.

- (a) Identify major datapath blocks used by ADD.
- (b) Identify additional blocks/control needed by LW.
- (c) If instruction memory=2 ns, register read=1 ns, ALU=3 ns, data memory=4 ns, register setup/write path allowance=1 ns, estimate critical instruction delay for ADD and LW using simple serial sums.
- (d) Which sets the clock period?

Worked solution: ADD: instruction fetch, decode/register file read, ALU, writeback mux/register write. Approx 2+1+3+1=7 ns. LW additionally uses immediate generation/ALU address, data memory and memory-to-register path: 2+1+3+4+1=11 ns. LW sets the single-cycle clock period, about 11 ns in this simplified model.

Problem 18 — Two-flop synchronizer reasoning [12 marks]

An asynchronous pushbutton enters a 100 MHz clock domain.

- (a) Why is direct sampling unsafe?
- (b) Draw/write a two-flop synchronizer.
- (c) Roughly how many destination cycles of latency does it introduce?
- (d) Why does it not solve switch bounce?

Worked solution: Direct sampling can violate setup/hold and cause metastability. Use two cascaded destination-clocked FFs with nonblocking assignments. The synchronized level typically appears after about two sampling edges (phase-dependent). Synchronization addresses metastability probability, not mechanical bounce; debouncing needs additional time/filter logic.

Problem 19 — Multi-bit CDC [12 marks]

An 8-bit counter value must cross from clkA to unrelated clkB.

- (a) Why is putting two synchronizer flops on each bit unsafe?
- (b) Give two safer approaches.
- (c) Why is Gray code often used for crossing counter pointers in asynchronous FIFOs?

Worked solution: Bits can be sampled from different source instants, creating incoherent words. Safer: source holds data stable and uses synchronized request/ack handshake; or asynchronous FIFO. Gray code changes only one bit between adjacent counts, reducing ambiguity when pointer values cross domains (with proper synchronization).

Problem 20 — Integrated RTL design: programmable pulse generator [20 marks]

Design a block with 50 MHz clk, synchronous reset, input ``period[15:0]``, and output ``pulse``. When enabled, pulse must be high for exactly one clock every ``period`` clock cycles.

- (a) Define behavior for period=0.
- (b) Determine counter comparison.
- (c) Write RTL.
- (d) List five verification cases.
- (e) If period=50000, what pulse repetition frequency results?

Worked solution: A robust specification can define period=0 as disabled (pulse=0, counter reset). Otherwise count 0...period-1; when count==period-1 assert pulse for one cycle and clear counter. Verification: reset, enable/disable if present, period=0, period=1 (pulse every clock), normal period, dynamic period change policy, max period. At 50 MHz / 50000 = 1 kHz.

Worked RTL for the Most Likely Coding Problems

1 ms tick from 50 MHz

```
logic [15:0] count;
always_ff @(posedge clk) begin
  if (rst) begin
    count <= 16'd0;
    tick <= 1'b0;
  end else begin
    tick <= 1'b0;
    if (count == 16'd49999) begin
      count <= 16'd0;
      tick <= 1'b1;
    end else
      count <= count + 1'b1;
  end
end
end
```

Saturating 8-bit counter

```
always_ff @(posedge clk) begin
  if (rst)
    count <= 8'h00;
  else if (en && count != 8'hFF)
    count <= count + 1'b1;
end
```

1011 overlapping Mealy detector

```
typedef enum logic [1:0] {S0,S1,S10,S101} state_t;
state_t state,next;

always_ff @(posedge clk) begin
  if (rst) state <= S0;
  else state <= next;
end

always_comb begin
  next = state;
  detect = 1'b0;
  case(state)
    S0: if (x) next=S1;
    S1: if (x) next=S1; else next=S10;
    S10: if (x) next=S101; else next=S0;
    S101: if (x) begin
      detect=1'b1; next=S1;
    end else next=S10;
  default: next=S0;
  endcase
end
```

Vending machine

```
typedef enum logic [1:0] {S0,S5,S10} state_t;
state_t state,next;

always_ff @(posedge clk)
  if(rst) state <= S0;
  else state <= next;

always_comb begin
  next=S0; dispense=0; change5=0;
  case(state)
    S0: begin
      if(coin5) next=S5;
      else if(coin10) next=S10;
      else next=S0;
    end
    S5: begin
      if(coin10) begin dispense=1; next=S0; end
      else if(coin5) next=S10;
      else next=S5;
    end
    S10: begin
      if(coin5) begin dispense=1; next=S0; end
      else if(coin10) begin

```

```
        dispense=1; change5=1; next=S0;
    end else next=S10;
end
default: next=S0;
endcase
end
```

Programmable pulse generator

```
logic [15:0] count;
always_ff @(posedge clk) begin
    if (rst || period==16'd0) begin
        count <= 16'd0;
        pulse <= 1'b0;
    end else if (count == period-1'b1) begin
        count <= 16'd0;
        pulse <= 1'b1;
    end else begin
        count <= count + 1'b1;
        pulse <= 1'b0;
    end
end
```

Four Full 60-Minute Mock Tests

These are intentionally only 4 substantial questions each. That forces you to do derivation, design and RTL rather than racing through definitions. Attempt each closed-book before reading the key.

Mock 1 — Digital + Timing + RTL

Q1 [15] Minimize $F(A,B,C,D)=\Sigma m(1,3,5,7,8,10,12,14)$. Draw the K-map, obtain minimal SOP, and show a NAND-only implementation.

Q2 [15] A register path has $T_{cq}=0.7$ ns, combinational max delay=6.1 ns, setup=0.8 ns, uncertainty=0.4 ns. Find T_{min} and f_{max} . If combinational delay is reduced by 25%, find new f_{max} .

Q3 [15] A 75 MHz clock must generate a one-cycle tick every 2 ms. Find modulus, minimum counter width, terminal count and write RTL.

Q4 [15] Given a faulty sequential block with two independent `if` statements for reset and enable, explain the conflict and rewrite a reset-priority saturating 8-bit counter.

Answer / marking guide

1. Q1: minterms split into $D=1$ with $A=0$? Work K-map carefully; full credit requires correct grouping and NAND-NAND mapping.
2. Q2: $T_{min}=0.7+6.1+0.8+0.4=8.0$ ns; $f_{max}=125$ MHz. New $T_{comb}=4.575$ ns; $T_{min}=6.475$ ns; $f_{max}\approx 154.44$ MHz.
3. Q3: $75e6 \times 0.002 = 150,000$ clocks. 17 bits because $2^{17}=131072 < 150000 < 2^{18}=262144$, so actually 18 bits. Terminal count 149,999. RTL counter resets and pulses at terminal count.
4. Q4: independent reset and enable can both execute; later assignment wins in simulation. Use `if(rst)... else if(en && count!=8'hFF)...` with nonblocking assignments.

Mock 2 — FSM + Waveform + Testbench

Q1 [20] Design an overlapping Mealy detector for 1101. Give states, complete transition table, detection points for input 1101101, and RTL.

Q2 [15] Design a 3-state IDLE/BUSY/DONE controller where start begins work, finish ends work, and done_pulse lasts exactly one cycle. Provide Moore outputs and RTL.

Q3 [10] Clock period=20 ns. Rising edges begin at 10 ns. Synchronous reset is high through the 10 ns edge and low from 13 ns. d changes 0->1 at 27 ns and 1->0 at 48 ns. Trace q at rising edges through 70 ns.

Q4 [15] Write a self-checking testbench plan/code for a modulo-10 counter. It must verify reset, enable hold, 0->9 sequence and 9->0 wrap.

Answer / marking guide

1. Q1: Prefix states S0,S1,S11,S110. Preserve useful suffix after detection; for 1101, detected final 1 leaves prefix S1. Input 1101101 contains detections ending at bits 4 and 7.
2. Q2: IDLE start->BUSY; BUSY finish->DONE; DONE->IDLE. busy only BUSY, done_pulse only DONE.
3. Q3: Rising edges 10,30,50,70. q=0 at10 reset; d=1 by30 so q=1; d=0 by50 so q=0; remains0 at70.
4. Q4: Expected-model counter, compare after each sampled update, explicitly check disabled hold and wrap.

Mock 3 — Broad D&V; Problem Solving

Q1 [15] A 6-bit signed adder computes $A+B$. For $A=25$ and $B=18$, represent operands/result in 6-bit two's complement and determine overflow. Repeat for $A=-20$, $B=-17$.

Q2 [15] A 4-stage ripple carry has 1.5 ns carry propagation per stage and 2.2 ns final carry-to-sum delay. Calculate worst C_4 and S_3 from C_0 , and the corresponding idealized maximum operation rate.

Q3 [15] An FSM controls a 15-unit vending machine accepting 5/10 and returning 5 change for 20. Give transition/output table and determine outputs for 5,10,10,10.

Q4 [15] Explain why synchronizing each bit of an 8-bit asynchronous bus with separate 2-FF chains is unsafe. Propose and sketch a handshake solution.

Answer / marking guide

1. Q1: 6-bit signed range -32..31. $25+18=43$ cannot be represented; binary wraps to 101011=-21, overflow. $-20 + -17=-37$ below -32; wraps to 011011=27, overflow.
2. Q2: $C_4=4 \times 1.5=6$ ns. $S_3=3 \times 1.5+2.2=6.7$ ns; max rate ≈ 149.25 MHz.
3. Q3: Use S_0, S_5, S_{10} ; outputs on completing payment. Sequence: 5 $\rightarrow S_5$; 10 dispenses $\rightarrow S_0$; 10 $\rightarrow S_{10}$; 10 dispenses+change $\rightarrow S_0$.
4. Q4: independent settling can create incoherent words. Source registers/stabilizes data, toggles/asserts req; destination synchronizes req, samples stable bus, then returns synchronized ack; source changes data only after ack.

Mock 4 — Final 60-Minute Challenge

Q1 [12] A 100 MHz system needs a 1 kHz one-clock enable pulse. Calculate divider modulus, counter width and terminal count.

Q2 [12] A timing path has $T_{cq}(\max)=0.9$ ns, logic=4.8 ns, setup=0.6 ns, uncertainty=0.2 ns. Calculate $f_{\max}$. Then check hold for $T_{cq}(\min)=0.2$ ns, logic(min)=0.1 ns, hold=0.45 ns with zero skew.

Q3 [18] Design a traffic controller with states NS_GREEN, NS_YELLOW, EW_GREEN, EW_YELLOW. A timer_done input advances each state. Give Moore outputs and RTL.

Q4 [18] A 4-bit counter RTL uses blocking assignments and has reset and load as separate if statements. Rewrite with priority reset > load > enable-count. Then write a compact self-checking testbench that verifies the priority when reset and load are asserted together.

Answer / marking guide

1. Q1: 100,000 clocks per 1 ms; modulus 100000; 17 bits since $2^{16} < 100000 < 2^{17}$; terminal 99999.
2. Q2: period=0.9+4.8+0.6+0.2=6.5 ns; $f_{\max} \approx 153.85$ MHz. Hold arrival=0.3 ns, required=0.45 ns, slack=-0.15 ns, violation by 0.15 ns.
3. Q3: timer_done advances cyclically NSG->NSY->EWG->EWY->NSG; outputs decoded solely from state. Include reset to NS_GREEN and safe default.
4. Q4: ``if(rst) q<=0; else if(load) q<=d; else if(en) q<=q+1;``. TB asserts rst=load=1 with nonzero d and checks q resets to 0, proving priority.

What to Prioritize in the Next 36 Hours

Tier 1: K-map/minimization; counters/dividers; setup/fmax and basic hold calculations; blocking/nonblocking tracing; latch/debugging; sequence-detector and controller FSMs; writing clean clocked/combinational RTL; simple self-checking testbenches.

Tier 2: signed overflow, ripple-carry timing, state encoding, processor datapath timing, synchronizers/CDC.

Tier 3: advanced UVM/SVA/coverage methodology. Know terminology, but do not sacrifice problem-solving practice for it before this exam.

Exam habit: show equations and assumptions. Even if your final RTL has a syntax slip, a correct state table, timing equation, K-map grouping or expected waveform can earn substantial partial credit.